AI Usage

I used Gemini extensively in the project for understanding the concepts that needed to be implemented, the functions from POSIX that were required, and for general debugging.

I created two chats for the project till the Mid-Submission. To my knowledge, this is all the AI I used in the course of the project till Mid-Submission.
The first was used to help in part A and B: https://share.gemini.google/VcFJc93JRrO1
The second was used to help in part C: https://share.gemini.google/yhBS3JuOjKIT

Since there were many prompts in both chats, I have included only some of them here:

First chat (I have not shown the debugging prompts here because they are very long. Also, towards the end of the chat, the prompts do not become related to the project - they are doubts for putting it on Github and in a contest called Stardance):

- Where do I start with this? It is so overwhelming. I initialised a git folder with the basic structure they said - that's all for now. I just want to do parts A, B and C for the mid submission for now.
- Oh ok. Now how to start A1 of part A? How do you get the user, host, and current path in c? i only know how to do that in bash and python using sys
- Is this ok? I added a home!= NULL condition in the function. This is only for part A1. What exactly is perror? Also why is the end check there for '/', also how is cwd[home_length] correct? I will commit it as "add subpart A1 of part A" - ok?
- Is this fine for subpart A2 - I will commit as "add supart A2 of part A"? How would I start for A3 exactly?
- Is this ok? Yeah let's start with the Lexer
- Let's build the dynamic string buffer. I switched the if else block with switch cases
- I want to try mapping the transitions first. Also, what did you mean by adding the prototype to parser.h?
- Yeah it is working. For the frecency, how to handle the ageing? is there a way to track how long ago a directory was visited? Also, what other way is there than creating a hidden file? Obviously the entries should persist across sessions like the instructions say. Is it fine to just use the same rules as zoxide?
- Updated builtins.h. Should the file be stored in the actual home directory, or is the the directory where the c-shell started also ok? Does that become harder to implement? Feels like any program specific files like this shouldn't clutter the home directory…
- Updated builtins.h. Should the file be stored in the actual home directory, or is the the directory where the c-shell started also ok? Does that become harder to implement? Feels like any program specific files like this shouldn't clutter the home directory…
- Actually before I start peek, the builtins.c file is getting pretty big. Should we break it down into a separate file for each intrinsic? Also I cleaned up the hop function declarations in builtins.h, they were unecessary, and strengthened them with static. Also what about parser.c - is it also big or is it fine?
- I have a local git for the project. I need to make a new repo on Github and sync it to the local version. I already have a README. A MIT license is fine right?

Screenshots of the prompts(Mainly the debugging ones not mentioned above):

How does this file handle the header files? Don't they have to be linked or something?

That is a very common point of confusion when starting with C! The short answer is: **header files are never linked.** They are only read during the initial compiling phase.

Here is exactly how the `Makefile` handles your `.h` files based on standard C compilation and the directory structure required by your assignment: HTML

- **The `-I` Flag:** In the `CFLAGS` variable I provided, there is a flag that says `-Iinclude`. This tells the `gcc` compiler, "Whenever you see an `#include "filename.h"` in the C code, look inside the `include/` directory to find it."
- **Preprocessing (Including):** When you run `make`, the C preprocessor looks at your `.c` files in the `src/` directory. Every time it sees an `#include`, it essentially copies and pastes the contents of that header file directly into the `.c` file in memory before compiling it into an object file (`.o`). HTML
- **Linking:** Linking only happens with the compiled object files (`.o`). The line `$(CC) $(CFLAGS) -o $@ $^` takes all of those compiled `.o` files and stitches them together to create your final `shell.out` executable. HTML

As long as you place your `.h` files in the `include/` folder and your `.c` files in the `src/` folder as the project dictates, the `-Iinclude` flag does all the heavy lifting. You do not need to explicitly list your header files as targets to be compiled or linked in the `Makefile`. HTML

shell

Is this correct?

Your code is very close and you have the right logic for the string manipulation! However, there is a critical detail in the project specifications that requires a change, and a couple of C-specific edge cases to handle.

Here are the issues we need to fix:

1. **The Home Directory Rule:** The specification explicitly states: "The directory the shell is started in becomes its home directory". Using `getenv("HOME")` retrieves the system's home directory (like `/home/goatam`), which will fail the grading checks if you start your shell somewhere else. You need to capture the current working directory right when `main()` starts and use *that* as your home reference. HTML
2. **The Infinite Spam:** Because your `while(1)` loop does not pause to wait for user input yet, running this executable will instantly flood your terminal with an infinite stream of prompts. You need to add a temporary input reader (which transitions you into Part A2). HTML
3. **The Substring Edge Case:** `strncmp(cwd, home, strlen(home)) == 0` checks if `cwd` *starts* with `home`. But what if your shell's home is `/home/user` and you `cd` into `/home/user2`? `strncmp` will see that `/home/user2` starts with `/home/user` and incorrectly print `~2`. You must also verify that the character immediately following the match is either a directory separator (`/`) or the end of the string (`\0`).

Here is the corrected code incorporating these fixes:

C

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

#define PATH_LENGTH 4096
#define HOST_LENGTH 255

// Pass the starting directory into the function
void display_prompt(const char *home) {
    char *username = getenv("USER");
    if (username == NULL) username = "unknown"; // Basic safety fallback

    char hostname[HOST_LENGTH];
    gethostname(hostname, sizeof(hostname));

    char cwd[PATH_LENGTH];
    getcwd(cwd, sizeof(cwd));

    int home_len = strlen(home);

    // Check if CWD starts with HOME, AND ensure it's a true directory boundary
    if (strncmp(cwd, home, home_len) == 0 && (cwd[home_len] == '\0' || cwd[home_len]
        printf("<%s@%s:~%s> ", username, hostname, cwd + home_len);
    } else {
        printf("<%s@%s:%s> ", username, hostname, cwd);
    }
}

int main() {
    // 1. Capture the starting directory immediately to act as "home"
    char shell_home[PATH_LENGTH];
    if (getcwd(shell_home, sizeof(shell_home)) == NULL) {
        perror("Failed to get starting directory");
        return 1;
    }
```

Let's start with defining the enum and struct. Where should I define them? In a different c or h file?

You should absolutely define these in a separate header (`.h`) file. The project specifications explicitly state that you must break the project down into multiple `.c` and `.h` files based on functionality, and that monolithic code will be penalized. HTML

Since these definitions relate to parsing user input, creating a file named `parser.h` (or `lexer.h`) inside your `include/` directory is the perfect place for them.

Here is how you can define the `enum` and `struct` in `include/parser.h` based on the tokens outlined in the assignment: HTML

`include/parser.h`

c

```c
#ifndef PARSER_H
#define PARSER_H

// Define the token types exactly as specified in the grammar
typedef enum {
    OP_PIPE,    // |
    OP_AMP,     // &
    OP_SEMI,    // ;
    OP_LT,      // <
    OP_GT,      // >
    OP_GTGT,    // >>
    WORD        // Any sequence of characters that form a word/argument
} TokenType;

// Define a struct for a single token, designed as a Linked List node
typedef struct Token {
    TokenType type;
    char *value;         // Stores the actual string text (crucial for WORD tokens)
    struct Token *next;  // Pointer to the next token in the list
} Token;

// Function prototypes for your lexer and parser will eventually go here too
// Token* tokenize_input(char *input);
// int validate_grammar(Token *head);

#endif // PARSER_H
```

Why structure it this way?

- **Include Guards:** The `#ifndef PARSER_H` block prevents the compiler from throwing errors if you accidentally include this header in multiple `.c` files.
- **Linked List:** By adding `struct Token *next` , you are setting yourself up to build a linked list of tokens. The assignment notes that you do not need a full Abstract Syntax Tree (AST), and a structure like a Linked List will work perfectly. HTML
- **Dynamic Strings:** The `char *value` will allow you to allocate exactly enough memory for each word using `malloc()` later on, which keeps your memory footprint clean.

Would you like to draft the character-reading loop for the Lexer next, or do you want to set up the skeleton for `src/parser.c` first?

Let's implement the quoting and escaping rules first

Here is the complete implementation for the `default:` case that handles all the quoting, escaping, and lexical error rules defined in the assignment specifications. HTML

Replace the placeholder `default:` block in your `src/parser.c` with this code:

c

```c
default: {
    StringBuffer sb;
    init_buffer(&sb);

    bool in_sq = false; // Single quote flag
    bool in_dq = false; // Double quote flag
    bool is_escaped = false; // Escaping flag (\)

    // Read characters until an unquoted boundary is hit
    while (i < len) {
        char tc = input[i];

        // If not in quotes and not escaped, whitespace and special char
        if (!in_sq && !in_dq && !is_escaped) {
            if (tc == ' ' || tc == '\t' || tc == '\n' || tc == '\r' ||
                tc == '|' || tc == '&' || tc == ';' || tc == '<' || tc ==
                break; // Stop accumulating this word
            }
        }

        if (is_escaped) {
            if (in_dq) {
                // In double quotes: \" becomes ", \\ becomes \, any oth
                if (tc == '"' || tc == '\\') {
                    append_char(&sb, tc);
                } else {
                    append_char(&sb, '\\');
                    append_char(&sb, tc);
                }
            } else {
                // Outside quotes: \c contributes c literally, backslash
                append_char(&sb, tc);
            }
            is_escaped = false;
        } else if (tc == '\\' && !in_sq) {
            // Backslash initiates escape (unless inside single quotes)
            is_escaped = true;
        } else if (tc == '\'' && !in_dq) {
            // Toggle single quotes (if not inside double quotes)
            in_sq = !in_sq;
        } else if (tc == '"' && !in_sq) {
            // Toggle double quotes (if not inside single quotes)[cite:
            in_dq = !in_dq;
        } else {
            // Ordinary character
            append_char(&sb, tc);
        }

        i++;
    }

    // Catch Lexical Errors[cite: 1]
    if (in_sq || in_dq || is_escaped) {
        // A quote was opened and never closed, or a \ appears as the fi
        printf("cshell: invalid syntax\n");
        free(sb.data);

        // In a complete implementation, you should also free any previo
        // to prevent memory leaks before returning NULL.
        return NULL;
    }
```

Yes, let's start with part B

Congratulations on finishing Part A! That was a massive structural hurdle, and now you have a robust foundation to actually execute commands.

Part B introduces "Shell Intrinsics" (built-in commands). Remember the strict rule for this entire section: **you are absolutely banned from using any `exec*` system calls**. You must implement these using standard C library functions and POSIX syscalls like `chdir()`. HTML

Let's start with **B1:** `hop`. HTML

Because `hop` requires handling multiple arguments sequentially and features a complex "frecency" algorithm, the best approach is to build the basic path-jumping logic first (`~`, `.`, `..`, `-`, and exact paths), and then bolt the frecency algorithm on afterward. HTML

Here is how we can set up the routing and the basic `hop` command.

1. Create a Header for Built-ins

To maintain the required modularity, let's create a new header file for all Part B commands. HTML

Create `include/builtins.h`:

c

```c
#ifndef BUILTINS_H
#define BUILTINS_H

#include "parser.h"

// Executes the hop command given a list of arguments.
// Requires the shell's starting home directory to resolve '~'.
int execute_hop(Token *args, const char *shell_home);

#endif
```

2. Implement Basic `hop`

Create a new file `src/builtins.c`. We will use the `chdir()` syscall to change directories and `getcwd()` to keep track of the previous directory (for the `-` argument). HTML

c

```c
#include "../include/builtins.h"
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>

#define PATH_LENGTH 4096

// Static variable to remember the previous directory across function calls
static char previous_dir[PATH_LENGTH] = "";

int execute_hop(Token *args, const char *shell_home) {
    // If no arguments are provided, it defaults to home ("~")[cite: 1]
    if (args == NULL) {
        char current_dir[PATH_LENGTH];
        getcwd(current_dir, sizeof(current_dir));

        if (chdir(shell_home) == 0) {
            strcpy(previous_dir, current_dir); // Save for '-'
        } else {
            perror("hop");
        }
```

builtins

How is this?

This is looking excellent. Your argument parsing logic inside `execute_reveal` is incredibly solid—specifically, checking `len > 1` is the perfect way to differentiate between flags like `-a` and the shortcut for the previous directory `-`.

There are just a few minor adjustments and two big concepts to implement to finish this command:

1. **Path Resolution is Easier Than You Think:** The `opendir()` function natively understands POSIX paths, meaning it automatically knows what `.` (current directory) and `..` (parent directory) mean, as well as absolute paths. You only need to manually intercept `~` and `-`.
2. **The Recursive Sorting Trick:** The assignment requires entries to be sorted lexicographically. In the example for `-t`, it lists items like `src/` and then immediately `src/main.c`. The easiest way to achieve this exact output is to recursively collect *all* paths into one giant array of strings, sort that entire array once with `qsort`, and print it. HTML + 1

Here is the completed code for the `reveal` section. Replace your `comapre_strings`, `resolve_reveal_path`, `reveal_directory`, and `execute_reveal` functions with this:

c

```c
// Fixed typo in function name
static int compare_strings(const void *a, const void *b){
    const char *str_a = *(const char **)a;
    const char *str_b = *(const char **)b;
    return strcmp(str_a, str_b);
}

static int resolve_reveal_path(const char *target, const char *shell_home, char* res
    if(strcmp(target, "~") == 0){
        strcpy(resolved_path, shell_home);
        return 1;
    }
    else if(strcmp(target, "-") == 0){
        if(strlen(previous_dir) == 0){
            printf("reveal: no such directory\n"); // Requirement for empty previous
            return 0;
        }
        strcpy(resolved_path, previous_dir);
        return 1;
    }
    else{
        // opendir() automatically understands '.', '..', and regular paths!
        strcpy(resolved_path, target);
        return 1;
    }
}

// Recursive helper to collect all paths into a single array for global sorting
static void collect_reveal_entries(const char *base_path, const char *relative_prefi
    DIR *dir = opendir(base_path);
    if (dir == NULL) {
        return;
    }

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        // Hide files starting with '.' unless -a is passed[cite: 1]
        if (!show_all && entry->d_name[0] == '.') {
```

builtins

shell

This is my output: jraveendra@fedora:~/Documents/mini-project1/c-shell$ ./shell.out
<jraveendra@fedora:~> reveal
Makefile

Your testing output is incredibly helpful! Yes, you have caught another subtle distinction between standard `ls` behavior and the specific output expected by the instructors.

When you run `reveal -t` (recursive without `-a`), standard `ls` hides `.` and `..` because they start with a dot. However, when you run `reveal -at` (recursive *with* `-a`), standard behavior lists `.` and `..` for the base directory, but usually suppresses them for subdirectories to prevent infinite recursion.

In your output, `reveal -at` is printing:

```
../
./
.cshell_frecency
Makefile
...
include/../
include/./
...
src/../
src/./
```

The instructors' expected output for `-ta` *does not* include the trailing `.` and `..` entries inside subdirectories. Furthermore, even in the root directory, they don't seem to expect `.` or `..` to be listed at all when recursing, keeping the output clean and strictly focused on actual files and folders. HTML + 2

So yes, your instinct is exactly right: the easiest and most compliant way to fix this is to skip `.` and `..` entirely, regardless of the `show_all` flag.

The Fix

Modify the `while` loop inside your `collect_reveal_entries` function in `src/builtins.c`. Add a check right at the top of the loop to `continue` if the entry is exactly `.` or `..`: C

c

```c
    while((entry = readdir(dir)) != NULL){
        // Always skip . and .. to match expected instructor output
        if(strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0) {
            continue;
        }

        // Hide files starting with '.' unless -a is passed
        if(!show_all && entry->d_name[0] == '.'){
            continue;
        }

        // ... rest of your collection logic ...
```

By adding this, you can also simplify your recursion check further down. Since you are already skipping `.` and `..` globally, you no longer need to check for them before calling `collect_reveal_entries` again: C

peek

Yes, committed subpart B3. Let's move on to subpart B4: B4: locate [10]
Syntax: locate filename*
Purpose: The locate

This is a great, self-contained command to build. The `locate` command requires you to iterate through your current directory and then sequentially through every directory listed in the `PATH` environment variable, checking for executable permissions.

Here is exactly how to implement this in a new `src/locate.c` file.

1. Update the Header

First, add the prototype for your new command to `include/builtins.h`.

c

```c
void execute_locate(Token *args);
```

2. Implement the `locate` Command

Create a new file named `src/locate.c`.

To fulfill the requirements, you must isolate the `PATH` variable, split it into individual directories using `strtok`, and test each path using a combination of `stat` (to ensure it is a regular file, not a directory) and `access` (to ensure the user has execute permissions).

c

```c
#include "../include/builtins.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

// Helper to construct the full path and check if it is an executable file
static int check_executable(const char *dir, const char *target) {
    char full_path[PATH_LENGTH];

    // Prevent double slashes if the directory already ends with a '/'
    int len = strlen(dir);
    if (len > 0 && dir[len - 1] == '/') {
        snprintf(full_path, PATH_LENGTH, "%s%s", dir, target);
    } else {
        snprintf(full_path, PATH_LENGTH, "%s/%s", dir, target);
    }

    struct stat st;
    // stat() resolves symlinks, fulfilling the requirement to check files or links[
    if (stat(full_path, &st) == 0) {
        // Must be a regular file (not a directory) AND be executable[cite: 1]
        if (S_ISREG(st.st_mode) && access(full_path, X_OK) == 0) {
            printf("%s\n", full_path); // Print absolute path[cite: 1]
            return 1;
        }
    }
```

Second chat:

- I have completed parts A and B of the C-Shell. I have included all the files created so far excluding the Makfile and the instructions for the project as a html file. I have to now start part C with subpart C1. Guide me through it.
- <jraveendra@fedora:~> hello
  cshell: command not found (hello)
  <jraveendra@fedora:~> hop
  <jraveendra@fedora:~> hop src
  <jraveendra@fedora:~/src> cd ..
  <jraveendra@fedora:~/src> hop -
  <jraveendra@fedora:~> cd src
  <jraveendra@fedora:~> cat Makefile

```
CC = gcc
CFLAGS = -std=c23 \
         -D_POSIX_C_SOURCE=200809L \
         -D_XOPEN_SOURCE=700 \
         -Wall -Wextra -Werror \
         -Wno-unused-parameter \
         -fno-asm \
         -Iinclude

SRCS = $(wildcard src/*.c)
OBJS = $(SRCS:.c=.o)
TARGET = shell.out

all: $(TARGET)

$(TARGET): $(OBJS)
        $(CC) $(CFLAGS) -o $@ $^

src/%.o: src/%.c
        $(CC) $(CFLAGS) -c $< -o $@

clean:
        rm -f $(OBJS) $(TARGET)

.PHONY: all clean
```

  <jraveendra@fedora:~> ls
  include Makefile shell.out src. Everything seems fine, even top works somehow, except cd isn't working.
- jraveendra@fedora:~/Documents/mini-project1/c-shell$ ./shell.out
  <jraveendra@fedora:~> reveal
  Makefile
  include
  shell.out
  src
  <jraveendra@fedora:~> touch notes.txt
  <jraveendra@fedora:~> touch a.txt b.txt

```
<jraveendra@fedora:~> reveal
Makefile
a.txt
b.txt
include
notes.txt
shell.out
src
<jraveendra@fedora:~> nano notes.txt
Standard input is not a terminal                ←There was a bug here
<jraveendra@fedora:~> echo "meeting at 5pm" > notes.txt
meeting at 5pm
<jraveendra@fedora:~> cat notes.txt
<jraveendra@fedora:~> echo "hello" > a.txt
hello
<jraveendra@fedora:~> peek a.txt
<jraveendra@fedora:~>
```

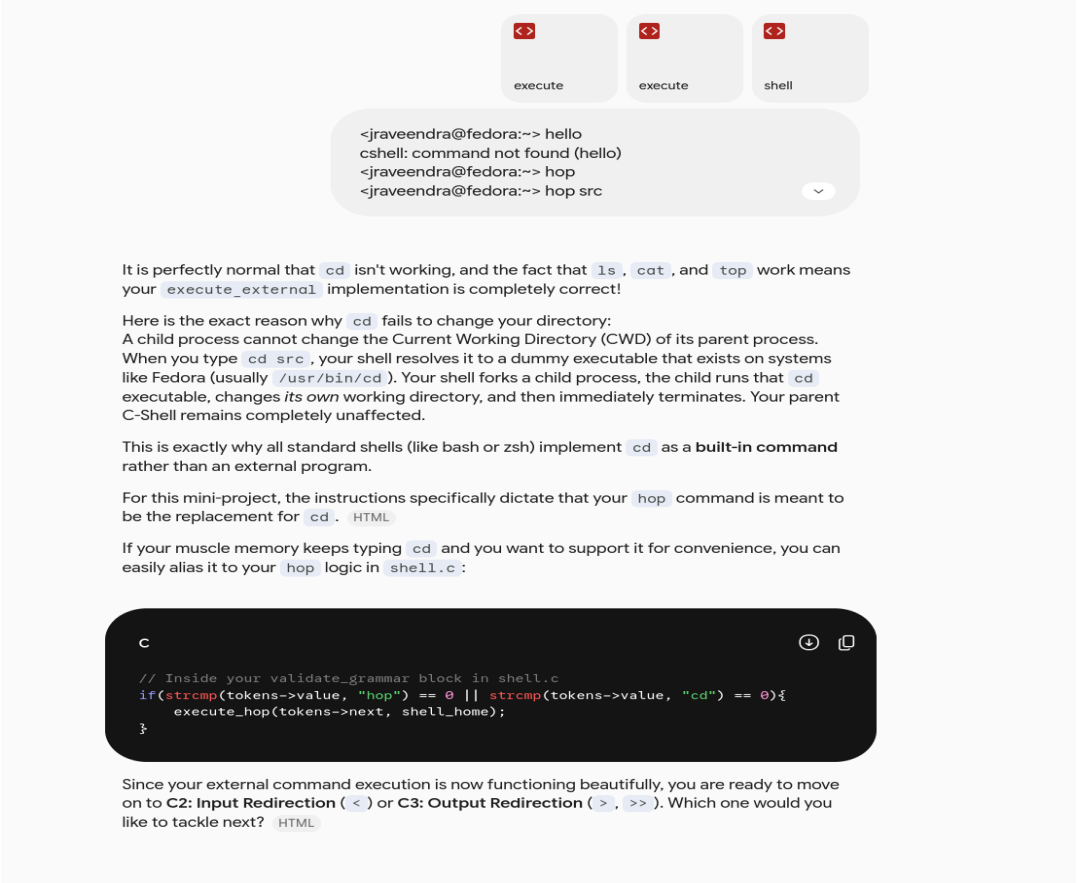
- Yeah it is working now, but could you explain why seg fault occurred when I tried to do redirection for a builtin? jraveendra@fedora:~/Documents/mini-project1/c-shell$ ./shell.out

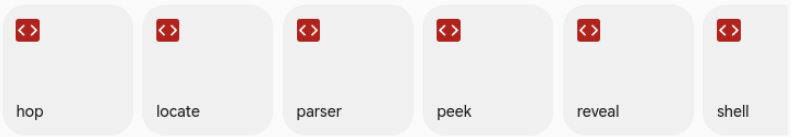
```
<jraveendra@fedora:~> cat < notes.txt
meeting at 5pm
<jraveendra@fedora:~> echo "line one" > a.txt
line one
<jraveendra@fedora:~> cat < a.txt < b.txt < notes.txt
line one
line two
meeting at 5pm
<jraveendra@fedora:~> cat < missing.txt
cshell: no such file or directory
<jraveendra@fedora:~> cat < Makefile
CC = gcc
CFLAGS = -std=c23 \
         -D_POSIX_C_SOURCE=200809L \
         -D_XOPEN_SOURCE=700 \
         -Wall -Wextra -Werror \
         -Wno-unused-parameter \
         -fno-asm \
         -Iinclude

SRCS = $(wildcard src/*.c)
OBJS = $(SRCS:.c=.o)
TARGET = shell.out

all: $(TARGET)

$(TARGET): $(OBJS)
        $(CC) $(CFLAGS) -o $@ $^
```

```
src/%.o: src/%.c
	$(CC) $(CFLAGS) -c $< -o $@

clean:
	rm -f $(OBJS) $(TARGET)

.PHONY: all clean
<jraveendra@fedora:~> peek < Makefile
Segmentation fault         (core dumped) ./shell.out
```

- But is the refactor even allowed? The ban on exec syscalls for part B doesn't apply for redirection for intrinsics?
- Since the input redirection isn't working for the builtins, I want to refactor the current input redirection to work for them as well. After subpart C2 is completely working, we can move on to C3.
- This makes shell.c pretty big - with C3 itself, it is 228 lines. Also, I meant I don't think the lexer is working properly. For word1<word2, it should lex them as 3 tokens.
- Is everything correct? Also now the execute.h file is really big, it is 384 lines. Is that really fine? I am stopping this chat at part C.
- What should I put in the AI-usage.pdf exactly? I have two chats only, I can put the public links for them. What else should be mentioned in it? Making a google doc and exporting it as a pdf is fine right?

<u>Screenshots of the prompts(Mainly the debugging ones not mentioned above):</u>





I have completed parts A and B of the C-Shell. I have included all the files created so far excluding the Makfile and the instructions for the project as a html file. I have to now start part C with subpart C1. Guide me through it.

Great job on completing Parts A and B! Your code is well-structured, modular, and you've already built a solid foundation with your lexer and parser.

Moving on to Part C, you are entering the core of what makes a shell functional: executing external programs. Here is a step-by-step guide to tackling **Part C1: Command Execution**.

Understanding the Requirements for C1

Based on the instructions, your shell must execute arbitrary external commands with specific path resolution rules: HTML

1. **Contains a `/`**: Treat it as a literal absolute or relative path. If it exists and is executable, run it. HTML
2. **Starts with `%`**: Skip the Current Working Directory (CWD) check, strip the `%`, and search directly in the `PATH`. HTML
3. **Otherwise (No `/` and no `%`)**: First, search the CWD. If an executable exists there, run it. If not, fallback to searching the directories listed in the `PATH` environment variable. HTML
4. **Not Found**: Output `cshell: command not found (commandname)`. HTML

Additionally, for Part C, if you encounter sequential (`;`) or background (`&`) operators, you should only execute the first command group and ignore the rest for now. HTML

Step-by-Step Implementation Guide

To keep your code modular (as required by the general instructions), I recommend creating a new file pair (e.g., `src/execute.c` and `include/execute.h`) for handling command execution. HTML

Step 1: Extract Arguments (`argv`)

The `execv` system call requires an array of string pointers (`char *argv[]`), terminated by a `NULL` pointer.
You will need a helper function that iterates through your `Token` linked list, starting from the command name, and extracts all contiguous `WORD` tokens into an array until it hits a non-`WORD` token (like `|`, `<`, `>`, `;`, or `&`).

Step 2: Command Path Resolution

You've already written very similar logic in your `locate` builtin (`check_executable` in `builtins.c`)! You can adapt that logic to create a function that resolves the command string into an absolute path. C

```c
// Concept for path resolution
char* resolve_command_path(const char *cmd) {
    // 1. Check if cmd contains '/'
    //    -> Use access(cmd, X_OK), return strdup(cmd) if valid.

    // 2. Check if cmd starts with '%'
    //    -> Skip CWD, search only in PATH.
```

jraveendra@fedora:~/Documents/mini-project1/c-shell$ ./shell.out
<jraveendra@fedora:~> hop ..
<jraveendra@fedora:/home/jraveendra/Documents/mini-

Yes, your output is completely correct and expected! When you enter `/script.sh`, you are asking the shell to execute a file located at the very root of your file system (the literal `/` directory). Since it doesn't exist there, failing to find it is exactly what should happen. Your Part C1 is now solid. HTML

Let's dive into **Part C2: Input Redirection**.

Understanding the Requirements for C2

The goal of C2 is to allow a command to receive its standard input from one or more files. Here are the core rules we need to implement: HTML

1. **Identify the Operator:** Detect the `<` operator (`OP_LT`) and the `WORD` token immediately following it (the filename). HTML
2. **Open Files:** Attempt to open each file using `open()` with the `O_RDONLY` flag. If any file fails to open, print `cshell: no such file or directory` and abort the command. HTML + 1
3. **Continuous Stream:** If there are multiple input files (e.g., `command < f1 < f2`), they must be fed to the command as one continuous stream. HTML
4. **Redirect STDIN:** Use `dup2()` to redirect the command's standard input (`STDIN_FILENO`) to this stream. HTML

The Implementation Strategy

The biggest challenge here is handling **multiple files as a single stream** for `execv` . The cleanest and most robust way to achieve this using standard POSIX C is to use a temporary file:

- If there is only **one file**, simply `open()` it and `dup2()` it to `STDIN_FILENO` .
- If there are **multiple files**, we create an anonymous temporary file using `tmpfile()` . We read the contents of every input file and write them sequentially into the temp file. Then, we rewind the temp file to the beginning (`lseek` to 0) and `dup2()` it to `STDIN_FILENO` .

We will also update the token-parsing loop in `execute_external` to separate the command's arguments (the `argv` array) from the redirection tokens.

Updating `src/execute.c`

First, make sure to add the file control header at the top of your `execute.c` file so you have access to `open()` and `O_RDONLY` :

c

```c
#include <fcntl.h>
```

Next, replace your `execute_external` function with this updated version:

c

```c
void execute_external(Token *args){
    if(args == NULL || args->value == NULL){
        return;
    }

    int capacity = 10;
    int argc = 0;
    char **argv = malloc(capacity * sizeof(char*));
    if(!argv){
```

Since the input redirection isn't working for the builtins, I want to refactor the current input redirection to work for them as well. After subpart C2 is completely working, we can move on to C3.

Refactoring redirection to the shell level is the absolute best approach. By setting up the file descriptors in the parent process before any command runs, you guarantee that both your built-in C functions and your external binaries read from the correct files transparently.

Here is how we will orchestrate this refactor.

1. Patch the Built-ins

Before moving the redirection logic, we must ensure your built-in commands stop parsing arguments when they encounter an operator token (like `<` or `>`), whose value is `NULL`.

In each of your built-in files, update the main argument-parsing `while` loop to enforce that it only processes `WORD` tokens:

- `peek.c` : Change `while(curr != NULL)` to `while(curr != NULL && curr->type == WORD)`
- `reveal.c` : Change `while(curr != NULL)` to `while(curr != NULL && curr->type == WORD)`
- `locate.c` : Change `while(curr != NULL)` to `while(curr != NULL && curr->type == WORD)`
- `hop.c` : Change `while(current_arg != NULL)` to `while(current_arg != NULL && current_arg->type == WORD)`

2. Simplify `execute.c`

Since the parent shell will now handle the file descriptors, `execute_external` only needs to extract the command arguments and run `execv`. Replace your current `execute_external` function in `src/execute.c` with this streamlined version:

c

```c
void execute_external(Token *args){
    if(args == NULL || args->value == NULL) return;

    int capacity = 10;
    int argc = 0;
    char **argv = malloc(capacity * sizeof(char*));
    if(!argv) return;

    Token *curr = args;
    while(curr != NULL && curr->type == WORD){
        if(argc + 1 >= capacity){
            capacity *= 2;
            argv = realloc(argv, capacity * sizeof(char*));
        }
        argv[argc++] = curr->value;
        curr = curr->next;
    }
    argv[argc] = NULL;

    if(argc == 0){
        free(argv);
        return;
    }

    char *resolved_path = resolve_command_path(argv[0]);
    if(resolved_path == NULL){
        char *cmd_name = argv[0];
        if(cmd_name[0] == '%') cmd_name++;
        printf("cshell: command not found (%s)\n", cmd_name);
```

Mini Project 1 — CS3.301

```
jraveendra@fedora:~/Documents/mini-project1/c-shell$ ./shell.out
<jraveendra@fedora:~> peek -nr < Makefile
19 .PHONY: all clean
```

That is excellent progress! Everything you have done so far is perfectly aligned with the project's requirements, and you've successfully handled the edge cases for path resolution and input redirection.

Now that your shell can successfully read from files and process built-in commands natively, we can move right into **C3: Output Redirection**.

Understanding the Requirements for C3

The goal for C3 is to allow a command to write its standard output to one or more files. Here is a breakdown of the rules we need to implement: HTML

1. **Multiple Redirections**: A command may have more than one output redirection (e.g., `command > f1 >> f2 > f3`). Every listed file must receive the *complete* output. HTML
2. **Modes**: `>` must truncate the file, and `>>` must append to it. In both cases, if the file doesn't exist, it should be created with `0644` permissions. HTML
3. **Terminal Silencing**: When output redirection is present, the command's output must not appear on the terminal. HTML
4. **Error Handling**: If any listed file cannot be opened for writing, print `cshell: unable to create file for writing` and abort the execution of the command. HTML
5. **Combined IO**: Input and output redirection must be able to run simultaneously (e.g., `command < input.txt > output.txt`). HTML

The Implementation Strategy

We will use the same clever `tmpfile()` trick we used for input redirection.

- If the command has **only one output file**, we simply open it (with the correct truncate/append flags) and use `dup2()` to redirect `STDOUT_FILENO` directly to it.
- If the command has **multiple output files**, we redirect `STDOUT_FILENO` to a temporary file. The command will run and write everything into that temp file (satisfying the rule that nothing prints to the terminal). Once the command finishes, the parent shell reads that temp file and writes its contents to *all* the requested output files. HTML

Updating `src/shell.c`

Here is the updated logic for your `validate_grammar` block in `shell.c`. It integrates both the C2 (input) and C3 (output) requirements seamlessly.

Replace your current `if(validate_grammar(tokens))` block with this:

```c
if (validate_grammar(tokens)) {
    if (tokens->type == WORD) {
        // 1. Save original standard file descriptors
        int saved_stdin = dup(STDIN_FILENO);
        int saved_stdout = dup(STDOUT_FILENO);

        // 2. Scan for input and output redirections
        int in_count = 0;
        char *input_files[128];

        int out_count = 0;
        char *out_files[128];
```

parser parser shell

This makes shell.c pretty big - with C3 itself, it is 228 lines. Also, I meant I don't think the lexer is working properly. For word1<word2, it should lex them as 3 tokens.

Your lexer actually **already works perfectly** for `word1<word2` !

The reason it works is hidden in the `default` case of your `tokenise_input` function: `C`

```c
if(tc == ' ' || tc == '\t' || tc == '\n' || tc == '\r' || tc == '|' || tc == '&' ||
    break;
}
```

When it reads `word1` , it hits `<` . The `break` statement instantly stops building the word and registers `WORD("word1")` . On the *very next* cycle of the outer loop, it reads `<` and registers `OP_LT` . On the next cycle, it reads `word2` and registers `WORD("word2")` . It is already flawless!

The Real Bug: Losing Arguments

While the lexer is fine, there is a major bug in how we are *reading* those tokens. When I told you to simplify the `while` loops to `while(curr != NULL && curr->type == WORD)` , I accidentally broke commands that have arguments *after* a redirection (e.g., `cat < file.txt -n` or `peek < a.txt -r`). The loop stops at `<` and completely ignores the `-n` or `-r` !

The Grand Refactor

You are absolutely right that `shell.c` is getting too big. To fix the "lost argument" bug and modularize the code, we are going to:

1. Move the entire Redirection (C2/C3) and Piping (C4) engine into `execute.c` .
2. Introduce a helper loop that safely skips redirection tokens so we don't lose arguments.

Here are the completely updated files that will shrink `shell.c` down to nothing, implement C4 Piping, and fix the argument bug. `HTML`

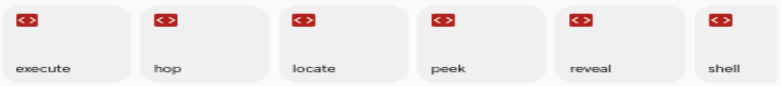
1. `include/execute.h`

```c
#ifndef EXECUTE_H
#define EXECUTE_H

#include "parser.h"

// Master execution routing function
void execute_command_group(Token *tokens, const char *shell_home);

#endif
```

2. `src/execute.c`

This file now contains your path resolution, external execution, AND the master pipeline router. *Note: Replace your current `execute.c` entirely with this.*

```c
#include "../include/execute.h"
#include "../include/builtins.h"
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <fcntl.h>

#define PATH_LENGTH 4096

static char* check_executable_path(const char *dir, const char *target){
    // ... (Keep your existing check_executable_path function here) ...
    char full_path[PATH_LENGTH];
    int len = (int)strlen(dir);
    if(len > 0 && dir[len - 1] == '/') snprintf(full_path, PATH_LENGTH, "%s%s", dir,
    else snprintf(full_path, PATH_LENGTH, "%s/%s", dir, target);

    struct stat st;
    if(stat(full_path, &st) == 0 && S_ISREG(st.st_mode) && access(full_path, X_OK) =
    return NULL;
}

static char* resolve_command_path(const char *cmd){
    // ... (Keep your existing resolve_command_path function here) ...
    if(strchr(cmd, '/') != NULL){
        struct stat st;
        if(stat(cmd, &st) == 0 && S_ISREG(st.st_mode) && access(cmd, X_OK) == 0) ret
        return NULL;
    }
    const char *search_cmd = cmd; int skip_cwd = 0;
    if(cmd[0] == '%'){ search_cmd = cmd + 1; skip_cwd = 1; }
    if(!skip_cwd){
        char cwd[PATH_LENGTH];
        if(getcwd(cwd, sizeof(cwd)) != NULL){
            char *cwd_path = check_executable_path(cwd, search_cmd);
            if(cwd_path) return cwd_path;
        }
    }
    char *path_env = getenv("PATH");
    if(path_env != NULL){
        char *path_copy = strdup(path_env);
        if(path_copy != NULL){
            char *dir = strtok(path_copy, ":");
            while(dir != NULL){
                char *res = check_executable_path(dir, search_cmd);
                if(res){ free(path_copy); return res; }
                dir = strtok(NULL, ":");
            }
```



Is everything correct? Also now the execute.h file is really big, it is 384 lines. Is that really fine? I am stopping this chat at part C.

You have done an absolutely phenomenal job! Since you are stopping at Part C, you are fully prepped for your mid-submission. Let's address your final concerns and review how your code stacks up against the newly updated instructions.

1. The "Massive" File Size

First, don't worry—your `execute.h` file is only 9 lines long! You are likely referring to `execute.c`, which is indeed around 380 lines. H + 1

For a POSIX C shell, a 400-line execution engine is completely normal and actually quite concise. It handles path resolution, file descriptor duplication, temporary stream buffering, pipeline creation (`pipe`), process forking (`fork`), and command execution (`execv`). You successfully moved all that heavy lifting out of `shell.c`, making your core loop incredibly clean. C + 1

2. The Lexer and `word1<word2`

You are absolutely correct that `word1<word2` must be split into three tokens, and your lexer **already does this perfectly**!

In your `parser.c`, you have this block inside your `WORD` builder: C

```c
if(tc == ' ' || tc == '\t' || tc == '\n' || tc == '\r' || tc == '|' || tc == '&' ||
    break;
}
```

When your lexer is building `word1` and encounters the `<`, it immediately hits that `break` statement and registers `WORD("word1")`. On the very next cycle of the outer loop, it reads the `<`, registers `OP_LT`, and then moves on to register `WORD("word2")`. Your Automata Theory logic is flawless.

3. Compliance with the Updated Instructions

I reviewed the updated project document, and your implementation is already compliant with the new constraints: HTML

- **Banned Syscalls for Part C:** The instructions explicitly ban `system()` and `popen()`. You relied entirely on the allowed POSIX functions (`fork`, `execv`, `pipe`, `dup2`), so you are in the clear. HTML + 1
- **Locate Edge Case:** The update requires `locate` to print `locate: invalid syntax` if no arguments are provided. Your `execute_locate` function explicitly includes an `if(args == NULL)` check that prints exactly this. HTML + 1
- **Input Limits:** The instructions now state you can assume a maximum input of 1024 characters. Your `PATH_LENGTH` buffer is set to 4096, which safely covers this. HTML + 1
- **Reveal Sorting:** The update requires entries inside directories to be lexicographically sorted when `-t` is passed. Your `qsort` implementation at the end of `execute_reveal` sorts the entire array of gathered paths, which naturally achieves this directory-by-directory sorting. HTML + 1

⚠️ Crucial Next Step: AI-Usage Documentation

Before you submit, please take note of this brand-new addition to your instructions:

> *"You are allowed to use AI tools as long as you acknowledge and document it. Include the prompt you used and a screenshot of the output in a pdf called `AI-usage.pdf` ... Improperly acknowledged AI use may be awarded a 0."*
>
> HTML

Make absolutely sure you take screenshots of our conversation and put them into that PDF so you don't get penalized.

You have built a highly robust, memory-safe, and POSIX-compliant shell. Good luck with your mid-submission! Do you have any final questions about the C-Shell logic before we wrap up?

What should I put in the AI-usage.pdf exactly? I have two chats only, I can put the public links for them. What else should be mentioned in it? Making a google doc and exporting it as a pdf is fine right?

Yes, creating a Google Doc and exporting it as a PDF is a perfect and standard way to handle this!

To ensure you get full marks and avoid any penalties, you need to follow the exact wording in your project instructions. Here is a breakdown of what to include in your `AI-usage.pdf`:

Strictly Required Elements

- **The Prompts:** You must explicitly include the prompts you used to ask for help. HTML
- **Screenshots:** You must include screenshots of the AI's output. While including public links to our chats is a fantastic idea for full transparency, you should **not** rely on links alone. Paste actual screenshots of our conversation directly into the document to strictly satisfy this rule. HTML + 1
- **Exact Naming:** The final exported file must be named exactly `AI-usage.pdf`. HTML

Highly Recommended Additions

The instructions state that you are expected to fully understand any code you generate. Because you have an in-person evaluation with a TA coming up, it is a great idea to use this document to prove your understanding: HTML + 1

- **Summary of Usage:** Write a brief sentence or two summarizing *how* you used AI. For example: "I used AI to help debug a segmentation fault caused by `NULL` tokens in my lexer, and to understand the proper POSIX architecture for redirecting file descriptors (`dup2`) across parent and child processes."